API Performance Report

Lyfflow backend (api.lyfflow.com) · production · measured 8 Nov 2026

58	1,334	253 ms	6 / 5	219 ms
GET endpoints measured	requests timed	median endpoint mean	critical / slow endpoints	network round trip

Summary

We timed every read-only (GET) endpoint the frontend calls that this account can reach: **58 endpoints**, 20 runs each on a reused connection plus 3 on a fresh one, **1,334 requests** in total, with no errors. **47 of 58** are healthy: the server spends under 500 ms on them, and most respond in about one network round trip (≈219 ms from the test machine). Nearly all the time users wait on those is network distance, not backend work.

The problems are concentrated in a few places. **6 endpoints are critical** (server time over 2 s) and **5 are slow** (0.5–2 s):

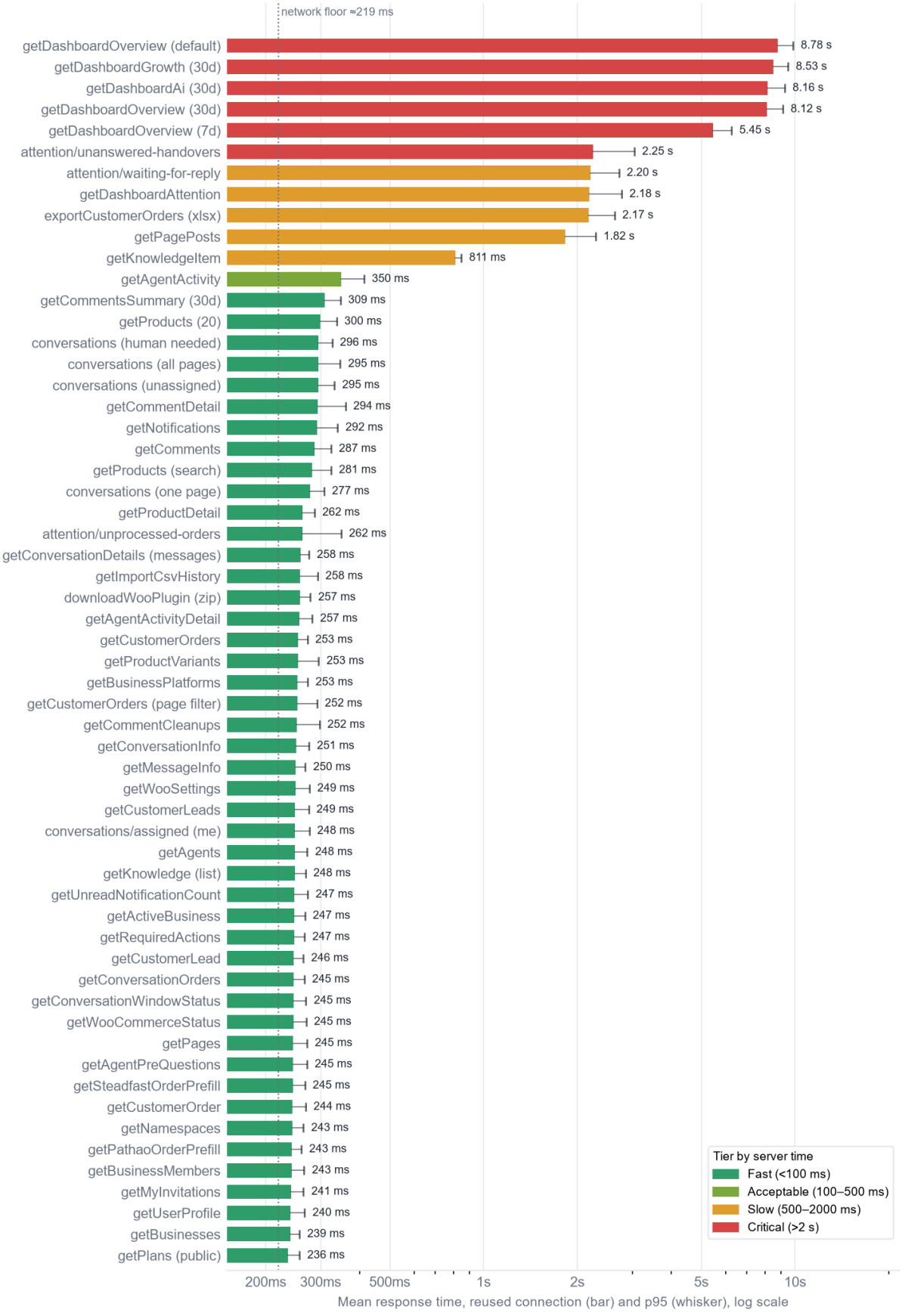
- **getDashboardOverview (default)** /v1/dashboard/overview: mean 8.78 s, p95 9.83 s, about 98% of it server time
- **getDashboardGrowth (30d)** /v1/dashboard/growth: mean 8.53 s, p95 9.48 s, about 97% of it server time
- **getDashboardAi (30d)** /v1/dashboard/ai: mean 8.16 s, p95 9.25 s, about 97% of it server time
- **getDashboardOverview (30d)** /v1/dashboard/overview: mean 8.12 s, p95 9.12 s, about 97% of it server time
- **getDashboardOverview (7d)** /v1/dashboard/overview: mean 5.45 s, p95 6.25 s, about 96% of it server time
- **attention/unanswered-handovers** /v1/dashboard/attention/unanswered-handovers: mean 2.25 s, p95 3.04 s, about 90% of it server time
- **attention/waiting-for-reply** /v1/dashboard/attention/waiting-for-reply: mean 2.20 s, p95 2.72 s, about 90% of it server time
- **getDashboardAttention** /v1/dashboard/attention: mean 2.18 s, p95 2.77 s, about 90% of it server time
- **exportCustomerOrders (xlsx)** /v1/pages/orders/export: mean 2.17 s, p95 2.63 s, about 69% of it server time
- **getPagePosts** /v1/business/pages/{id}/posts: mean 1.82 s, p95 2.29 s, about 76% of it server time
- **getKnowledgeItem** /v1/knowledge/{id}/{id}: mean 811 ms, p95 848 ms, about 73% of it server time

The **owner dashboard** is the biggest problem. It is the first screen an owner sees, and its overview, AI, growth and attention calls are among the slowest endpoints in the system. Section 4 covers likely causes and fixes. In short: precompute daily aggregates, cache per business for a short time, and keep the Facebook posts call off the request path.

Tiers are based on **estimated server time** rather than total time, so the network distance from the test machine doesn't make an endpoint look slow. Fast: under 100 ms · Acceptable: 100–500 ms · Slow: 0.5–2 s · Critical: over 2 s.

1. Ranking: slowest at the top, fastest at the bottom

Each bar is the mean over a reused (keep-alive) connection, which is how the browser makes most calls. The whisker runs to the 95th percentile. The dotted line is the network round trip, the minimum any endpoint can take from here.



2. Full results

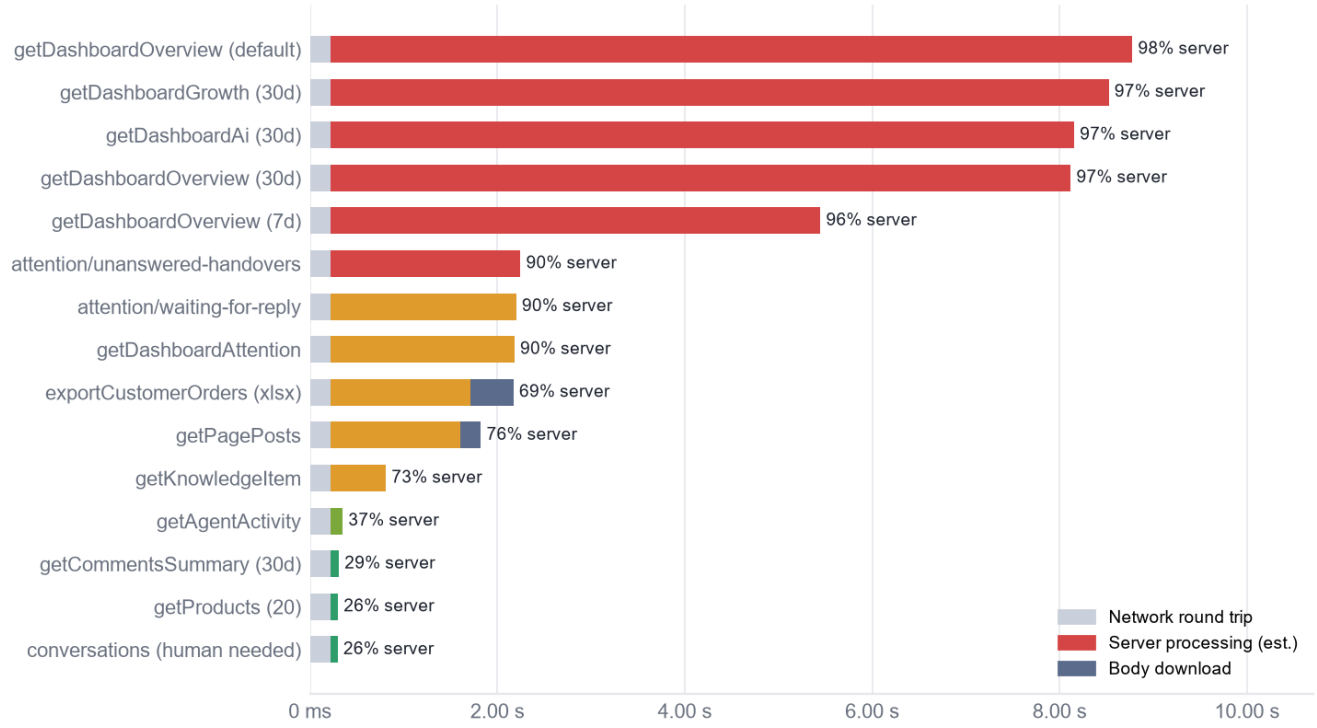
All times in ms, reused connection, n=20 per endpoint unless noted. **Server** = time to first byte minus the request send and one round trip (the backend's own processing). **Cold** = mean of 3 requests on a brand-new connection (adds DNS, TCP and TLS), which is what the first call after a page load or a long idle pays.

#	Endpoint	Area	Mean	Median	p90	p95	Max	Min	SD	Server	Cold	Size	Tier
1	**getPlans (public)** `/v1/plans`	Account	**236**	238	253	256	276	189	17	19	779	901 B	Fast
2	**getBusinesses** `/v1/businesses`	Account	**239**	238	254	256	267	198	15	21	784	223 B	Fast
3	**getUserProfile** `/v1/user/profile`	Account	**240**	237	259	267	268	197	17	22	727	170 B	Fast
4	**getMyInvitations** `/v1/business/invitations`	Account	**241**	242	264	264	267	194	18	23	678	2 B	Fast
5	**getBusinessMembers** `/v1/business/members`	Business	**243**	244	260	265	271	204	15	24	764	1.3 KB	Fast
6	**getPathaoOrderPrefill** `/v1/pathao/orders/{id}/prefill`	Integrations	**243**	248	258	260	268	194	17	25	709	391 B	Fast
7	**getNamespaces** `/v1/knowledge/get-namespaces`	Knowledge	**243**	244	264	264	266	196	18	25	745	102 B	Fast
8	**getCustomerOrder** `/v1/pages/orders/{id}`	Orders	**244**	242	267	268	291	193	21	26	671	812 B	Fast
9	**getSteadfastOrderPrefill** `/v1/steadfast/orders/{id}/prefill`	Integrations	**245**	242	262	267	268	205	15	26	714	382 B	Fast
10	**getAgentPreQuestions** `/v1/agent/on-boarding/pre-questions`	Agents	**245**	243	265	271	307	214	21	26	686	1.0 KB	Fast
11	**getPages** `/v1/business/pages`	Business	**245**	243	268	271	289	194	21	27	741	658 B	Fast
12	**getWooCommerceStatus** `/v1/woocommerce/status`	Integrations	**245**	243	264	269	287	192	19	27	698	371 B	Fast
13	**getConversationWindowStatus** `/v1/page/{id}/conversations/{id}/window-status`	Inbox	**245**	242	265	269	295	197	19	27	756	290 B	Fast
14	**getConversationOrders** `/v1/pages/orders/conversation/{id}`	Orders	**245**	251	262	266	278	196	18	27	718	814 B	Fast
15	**getCustomerLead** `/v1/pages/leads/{id}`	Leads	**246**	245	260	264	275	209	14	27	762	822 B	Fast
16	**getRequiredActions** `/v1/user-required-actions`	Account	**247**	256	265	266	277	189	21	29	726	14 B	Fast
17	**getActiveBusiness** `/v1/business`	Account	**247**	250	264	267	297	199	20	29	706	542 B	Fast
18	**getUnreadNotificationCount** `/v1/notifications/unread-count`	Notifications	**247**	249	270	273	277	201	19	29	725	21 B	Fast
19	**getKnowledge (list)** `/v1/knowledge/{id}`	Knowledge	**248**	246	267	268	272	203	18	29	788	655 B	Fast
20	**getAgents** `/v1/agents`	Agents	**248**	247	268	272	309	201	22	30	764	5.0 KB	Fast
21	**conversations/assigned (me)** `/v1/conversations/assigned`	Inbox	**248**	248	271	276	285	204	19	30	651	96 B	Fast
22	**getCustomerLeads** `/v1/pages/leads`	Leads	**249**	252	271	274	285	209	18	30	762	3.1 KB	Fast
23	**getWooSettings** `/v1/woocommerce/settings`	Integrations	**249**	248	275	276	283	197	20	31	638	57 B	Fast
24	**getMessageInfo** `/v1/conversations/{id}/messages/{id}/info`	Inbox	**250**	250	265	268	275	203	16	31	757	825 B	Fast
25	**getConversationInfo** `/v1/conversations/{id}/info`	Inbox	**251**	252	272	275	280	206	17	32	727	463 B	Fast
26	**getCommentCleanups** `/v1/comments/cleanups`	Comments	**252**	249	287	298	329	195	29	34	770	87 B	Fast
27	**getCustomerOrders (page filter)** `/v1/pages/orders`	Orders	**252**	251	278	292	292	205	20	34	781	4.4 KB	Fast
28	**getBusinessPlatforms** `/v1/business/platforms`	Business	**253**	255	270	273	323	202	24	35	704	596 B	Fast
29	**getProductVariants** `/v1/products/{id}/{id}/variants`	Products	**253**	249	287	296	356	194	31	35	813	2 B	Fast

#	Endpoint	Area	Mean	Median	p90	p95	Max	Min	SD	Server	Cold	Size	Tier
30	**getCustomerOrders** `/v1/pages/orders`	Orders	**253**	254	270	272	274	201	17	35	699	4.4 KB	Fast
31	**getAgentActivityDetail** `/v1/agent/{id}/agent_activity/{id}/detail`	Agents	**257**	259	277	282	320	201	25	38	730	1.5 KB	Fast
32	**downloadWooPlugin (zip)** `/v1/woocommerce/plugin/download`	Integrations	**257**	251	267	278	378	233	30	26	1093	60.9 KB	Fast
33	**getImportCsvHistory** `/v1/products/{id}/import/csv/history`	Products	**258**	253	287	293	316	205	23	40	732	90 B	Fast
34	**getConversationDetails (messages)** `/v1/page/{id}/conversation/{id}`	Inbox	**258**	260	272	275	305	220	17	39	769	6.1 KB	Fast
35	**attention/unprocessed-orders** `/v1/dashboard/attention/unprocessed-orders`	Dashboard	**262**	253	279	349	359	225	34	43	695	88 B	Fast
36	**getProductDetail** `/v1/products/{id}/detail/{id}`	Products	**262**	261	278	287	328	209	23	44	718	3.8 KB	Fast
37	**conversations (one page)** `/v1/conversations`	Inbox	**277**	279	298	308	314	230	19	58	760	6.0 KB	Fast
38	**getProducts (search)** `/v1/products/{id}/all-products`	Products	**281**	280	311	324	350	232	28	62	867	14.5 KB	Fast
39	**getComments** `/v1/comments`	Comments	**287**	287	319	324	329	250	20	66	1025	37.9 KB	Fast
40	**getNotifications** `/v1/notifications`	Notifications	**292**	293	327	340	352	214	32	73	908	13.8 KB	Fast
41	**getCommentDetail** `/v1/comments/{id}`	Comments	**294**	259	289	360	943	230	154	75	749	2.6 KB	Fast
42	**conversations (unassigned)** `/v1/conversations`	Inbox	**295**	287	325	332	367	243	28	75	778	6.2 KB	Fast
43	**conversations (all pages)** `/v1/conversations`	Inbox	**295**	287	335	347	354	242	27	76	765	6.3 KB	Fast
44	**conversations (human needed)** `/v1/conversations`	Inbox	**296**	282	309	326	575	227	69	76	744	5.8 KB	Fast
45	**getProducts (20)** `/v1/products/{id}/all-products`	Products	**300**	293	334	337	392	263	30	78	1138	40.0 KB	Fast
46	**getCommentsSummary (30d)** `/v1/comments/summary`	Comments	**309**	300	345	347	379	273	27	90	767	3.2 KB	Fast
47	**getAgentActivity** `/v1/agent/{id}/agent_activity`	Agents	**350**	342	393	413	478	304	40	130	783	3.5 KB	Acceptable
48	**getKnowledgeItem** `/v1/knowledge/{id}/{id}`	Knowledge	**811**	812	842	848	855	773	25	591	1289	937 B	Slow
49	**getPagePosts** `/v1/business/pages/{id}/posts`	Business	**1824**	1697	2246	2289	2343	1456	300	1386	2351	44.3 KB	Slow
50	**exportCustomerOrders (xlsx)** `/v1/pages/orders/export`	Orders	**2172**	2138	2578	2630	2632	1763	298	1497	2682	166.2 KB	Slow
51	**getDashboardAttention** `/v1/dashboard/attention`	Dashboard	**2183**	2113	2723	2773	2808	1396	345	1964	2595	3.0 KB	Slow
52	**attention/waiting-for-reply** `/v1/dashboard/attention/waiting-for-reply`	Dashboard	**2201**	2183	2619	2718	2813	1754	300	1982	2589	3.0 KB	Slow
53	**attention/unanswered-handovers** `/v1/dashboard/attention/unanswered-handovers`	Dashboard	**2246**	2035	2864	3044	3064	1761	393	2027	2721	6.0 KB	Critical
54	**getDashboardOverview (7d)** `/v1/dashboard/overview`	Dashboard	**5448**	5391	5914	6246	7169	4220	607	5229	5520	2.3 KB	Critical
55	**getDashboardOverview (30d)** `/v1/dashboard/overview`	Dashboard	**8119**	8101	9021	9125	9363	7188	682	7900	9278	4.6 KB	Critical
56	**getDashboardAi (30d)** `/v1/dashboard/ai`	Dashboard	**8156**	8191	9187	9252	10052	7049	822	7937	8901	3.8 KB	Critical
57	**getDashboardGrowth (30d)** `/v1/dashboard/growth`	Dashboard	**8525**	8467	9382	9482	10416	7693	672	8306	7998	2.1 KB	Critical
58	**getDashboardOverview (default)** `/v1/dashboard/overview`	Dashboard	**8777**	8613	9784	9833	10217	7939	634	8558	9238	4.6 KB	Critical

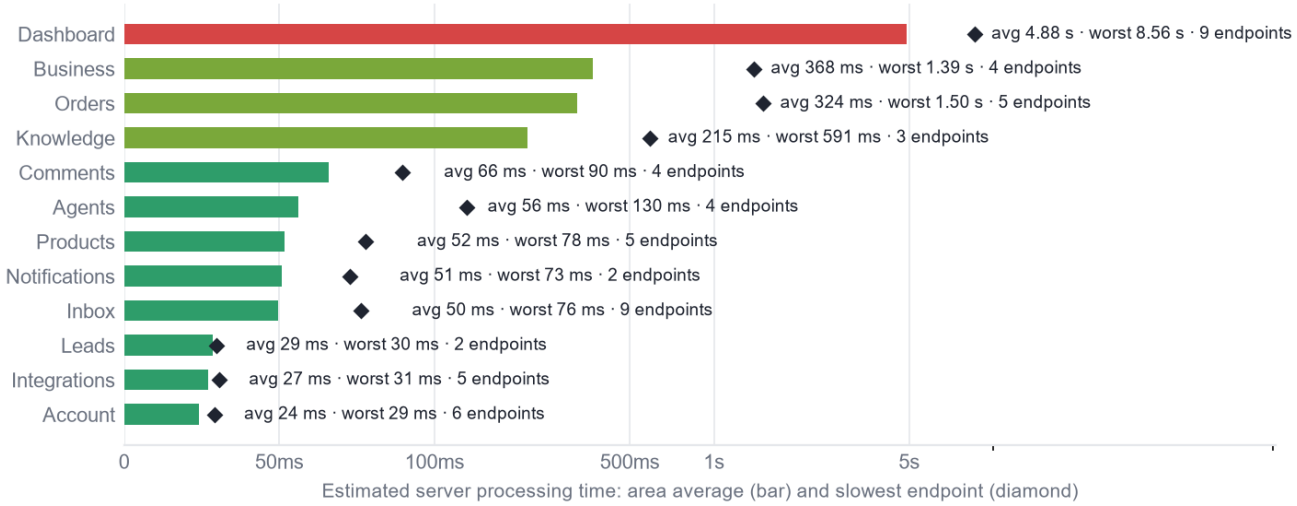
3. Where the time goes

For the 15 slowest endpoints, the bar splits total time into network round trip, server processing and body download. Where the colored segment dominates, the fix belongs in the backend: query, aggregation or an upstream call.



By feature area

Average estimated server time per area, with the slowest endpoint in each area marked.



4. Endpoints that need improvement

Ordered by impact: how slow each one is, weighted by how often users hit it. We could only observe these from outside (the backend code isn't in this repo), so causes are inferred from the timing pattern and the response shape. Confirm them with server-side profiling before changing anything.

4.1 Owner dashboard aggregates (Critical, highest priority)

getDashboardOverview (default) 8.78 s (p95 9.83 s, server 8.56 s) · **getDashboardOverview (30d)** 8.12 s (p95 9.12 s, server 7.90 s) · **getDashboardOverview (7d)** 5.45 s (p95 6.25 s, server 5.23 s) · **getDashboardAi (30d)** 8.16 s (p95 9.25 s, server 7.94 s) · **getDashboardGrowth (30d)** 8.53 s (p95 9.48 s, server 8.31 s)

These run on every dashboard visit and on every period change. Server time is nearly the whole response. The overview takes 5.45 s for 7 days and 8.12 s for 30. If time grew linearly with the range, that would be roughly 116 ms of server time per extra day on top of a ~4415 ms fixed cost. That pattern fits per-day or per-conversation queries run in a loop, or scans over raw messages, rather than reads of pre-aggregated data. The dashboard shows one tab at a time, but the default 30-day overview alone keeps an owner waiting several seconds on first load.

- Keep a **daily rollup table** per business and page (conversations, messages, AI replies, orders, revenue, handovers), updated by the worker or a scheduled job. Dashboard reads then become a sum over at most 30 rows.
- Until that exists, **cache each (business, range) response** server-side for 60–120 s (Redis), and send `Cache-Control: private, max-age=60` so the browser can reuse it too.
- Check that the queries behind these are covered by composite indexes on (business_id/page_id, created_at). Run EXPLAIN ANALYZE on the overview query first.
- If the endpoint runs several independent counts one after another, run them concurrently (asyncio.gather) or fold them into one grouped query.
- Frontend: render the other dashboard sections while overview loads (skeleton states), and prefetch the 30-day view right after login.

4.2 Attention queues (Critical/Slow)

getDashboardAttention 2.18 s (p95 2.77 s, server 1.96 s) · **attention/unanswered-handovers** 2.25 s (p95 3.04 s, server 2.03 s) · **attention/waiting-for-reply** 2.20 s (p95 2.72 s, server 1.98 s)

These are labelled "always live" in the frontend, so they are never cached. 'unprocessed-orders' takes 262 ms, so the slow part is working out which conversations are waiting. That probably means comparing each conversation's latest customer message with the latest reply, which is expensive without a stored state.

- Store **waiting_since** / **last_customer_message_at** / **last_reply_at** on the conversation row when messages are written, and index (page_id, waiting_since). The queue then becomes a simple indexed range read.
- The summary endpoint (/dashboard/attention) can count from the same columns. Avoid building the preview lists and the counts in separate passes.
- A 10–15 s server cache is fine for counts, even for a 'live' view.

4.3 Facebook page posts (Slow)

getPagePosts 1.82 s (p95 2.29 s, server 1.39 s)

This response is large and its time looks like a live call to the Meta Graph API on every request (latency we don't control, plus the body download).

- Cache posts per page for a few minutes, or sync them in the background and serve them from the database.
- Ask Graph only for the fields the comments screen shows, and keep the page size small (the frontend requests 25).

4.4 Order export (Slow, acceptable for a download)

exportCustomerOrders (xlsx) 2.17 s (p95 2.63 s, server 1.50 s)

Generating the spreadsheet takes a few seconds and the file is about 170 KB. For an on-demand download that's tolerable, but it grows with order volume.

- Write rows to the spreadsheet in streaming mode (openpyxl write_only) and load orders and their items in one batched query, not one query per order.
- Once a month of orders takes over ~5 s, move the export to a background job and notify the user when the file is ready.

4.5 Other slow endpoints

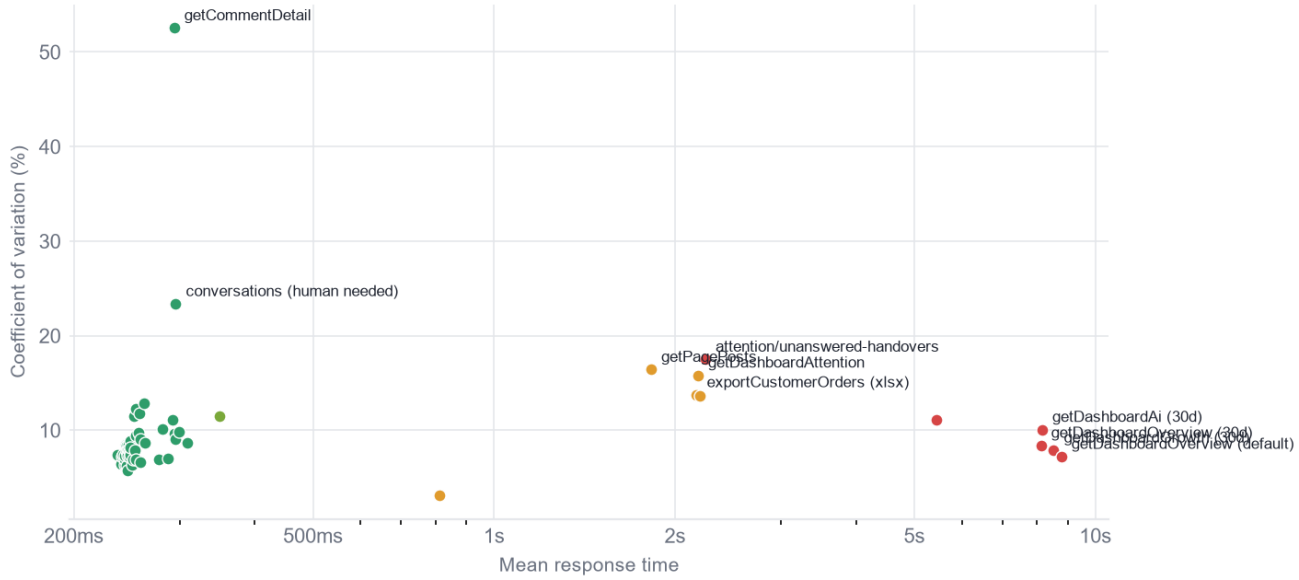
- **getKnowledgeItem** `/v1/knowledge/{id}/{id}`: mean 811 ms, p95 848 ms, server 591 ms, 937 B. The knowledge list returns in about 30 ms of server time, but one item takes about 600 ms. The item body is probably fetched from object storage or the vector store on each read. Store the text in the database row or cache it, and load it only when the editor opens.

4.6 Across the whole API

- **Connection setup is expensive.** A fresh connection costs about 479 ms (TCP 239 ms + TLS 240 ms) before any request is sent. So the first call after a page load is roughly twice as slow as later ones. The server only negotiates HTTP/1.1 (we checked ALPN), so a browser opens up to 6 parallel connections and pays this cost on each. Enable HTTP/2 on nginx (`listen 443 ssl; http2 on;`) so every call shares one connection, keep keepalive_timeout long, and enable TLS session resumption.
- **Distance dominates fast endpoints.** Most endpoints spend under 100 ms on the server but take ≈219 ms in total from here. If most users are in Bangladesh, check where the API server is hosted. A region closer to them (e.g. Singapore/Mumbai) would cut every call roughly in half.
- **Responses are not compressed and carry no Cache-Control.** We asked for gzip/br and got plain JSON back (e.g. /v1/comments: 38 KB uncompressed). Turn on gzip/brotli in nginx for application/json responses. The large lists (products, comments, posts, notifications) are 14–45 KB and very repetitive, so they would compress 5–10×.
- **Add server-side timing.** Send a `Server-Timing` header (db, upstream, total) and log per-route latency percentiles, so this report can come from real traffic instead of synthetic tests.

5. Consistency

Coefficient of variation (standard deviation ÷ mean) shows how unpredictable an endpoint is. Fast endpoints with a high CV are usually just network jitter. Slow endpoints with a high CV point to lock contention, cold caches or a variable upstream call.



Least consistent endpoints

Endpoint	Mean	SD	CV	Min	Max	Max ÷ median
getCommentDetail	294 ms	154 ms	53%	230 ms	943 ms	3.6×
conversations (human needed)	296 ms	69 ms	23%	227 ms	575 ms	2.0×
attention/unanswered-handovers	2.25 s	393 ms	17%	1.76 s	3.06 s	1.5×
getPagePosts	1.82 s	300 ms	16%	1.46 s	2.34 s	1.4×
getDashboardAttention	2.18 s	345 ms	16%	1.40 s	2.81 s	1.3×
exportCustomerOrders (xlsx)	2.17 s	298 ms	14%	1.76 s	2.63 s	1.2×
attention/waiting-for-reply	2.20 s	300 ms	14%	1.75 s	2.81 s	1.3×
attention/unprocessed-orders	262 ms	34 ms	13%	225 ms	359 ms	1.4×

6. Not measured

The frontend calls 158 distinct endpoints. This report covers the read-only ones this account can reach. The rest were left out on purpose:

- **70 write endpoints** (POST/PATCH/DELETE). Running them on production would create or change real orders, send Messenger replies, spend comment-cleanup credits and so on. Measure these on a staging copy, or from production access logs.
- **33 platform-admin endpoints** (/v1/admin/*): they need an admin session, and this one returned 401.
- **Owner-only**: /v1/subscription and /v1/api-keys returned 403 for this account's role (business admin, not owner).
- **Pathao** stores and locations: Pathao isn't connected for this business (400). **Steadfast/Pathao prefill** were measured because they read local order data.
- **/v1/notifications/stream** is a long-lived SSE stream, so response time doesn't apply to it.
- **POST /v1/chat**, **/agent/on-boarding/pre-instructions** and **/comments/cleanups/estimate** are POSTs that call the LLM or need a body, so they're excluded for the same reason as the writes.

Area	Write endpoints not measured

agent	`DELETE /v1/agent/delete`
	`PATCH /v1/agent/page/{id}/conversation/{id}/pause`
	`PATCH /v1/agent/update/{id}`
	`PATCH /v1/agent/{id}/set-avatar`
	`PATCH /v1/agent/{id}/set-namespace/{id}`
	`PATCH /v1/agent/{id}/unset-namespace`
	`POST /v1/agent/create`
	`POST /v1/agent/on-boarding/pre-instructions`

api-keys	`DELETE /v1/api-keys/{id}` `POST /v1/api-keys/create`

business	`DELETE /v1/business`
	`DELETE /v1/business/members/{id}`
	`DELETE /v1/business/pages/{id}`
	`PATCH /v1/business`
	`PATCH /v1/business/members/{id}`
	`PATCH /v1/business/pages/{id}/assign-agent`
	`PATCH /v1/business/pages/{id}/posts/settings`
	`PATCH /v1/business/pages/{id}/unassign-agent`
	`POST /v1/business/members/accept`
	`POST /v1/business/members/invite`
	`POST /v1/business/members/reject`

businesses	`POST /v1/businesses` `POST /v1/businesses/{id}/switch`
chat	`POST /v1/chat`

comments	`POST /v1/comments/bulk-action`
	`POST /v1/comments/cleanups`
	`POST /v1/comments/cleanups/estimate`
	`POST /v1/comments/cleanups/{id}/cancel`
	`POST /v1/comments/{id}/action`

conversations	`DELETE /v1/conversations/{id}/attachments/{id}/visual-label/{id}` `POST /v1/conversations/{id}/attachments/{id}/visual-label`
facebook	`POST /v1/facebook/{id}/messenger/{id}/reply` `POST /v1/facebook/{id}/messenger/{id}/reply/file`
feedback	`POST /v1/feedback`

knowledge	`DELETE /v1/knowledge/{id}/{id}`
	`PATCH /v1/knowledge/{id}/{id}`
	`POST /v1/knowledge/generate-namespace`
	`POST /v1/knowledge/{id}/create`
	`POST /v1/knowledge/{id}/upload`

leads	`POST /v1/leads/create`
notifications	`DELETE /v1/notifications/{id}` `POST /v1/notifications/read`
page	`PATCH /v1/page/{id}/conversations/{id}/assign` `PATCH /v1/page/{id}/conversations/{id}/note`

pages	`DELETE /v1/pages/orders/{id}`
	`PATCH /v1/pages/leads/{id}/status`
	`PATCH /v1/pages/orders/{id}`
	`PATCH /v1/pages/orders/{id}/status`

pathao	`POST /v1/pathao/connect`
	`POST /v1/pathao/orders/{id}/place`
	`POST /v1/pathao/price-plan`

Area	Write endpoints not measured

products	`DELETE /v1/products/{id}/delete/{id}`
	`DELETE /v1/products/{id}/update/{id}/delete-assets/{id}`
	`DELETE /v1/products/{id}/{id}/variants/{id}`
	`PATCH /v1/products/{id}/update/{id}`
	`PATCH /v1/products/{id}/update/{id}/set-assets/primary`
	`PATCH /v1/products/{id}/{id}/variants/{id}`
	`POST /v1/products/{id}/create`
	`POST /v1/products/{id}/import/csv`
	`POST /v1/products/{id}/update/{id}/add-assets`
	`POST /v1/products/{id}/{id}/variants`

profile	`PATCH /v1/profile/update`
steadfast	`POST /v1/steadfast/connect` `POST /v1/steadfast/orders/{id}/place`
subscription	`POST /v1/subscription/subscribe`
user-required-actions	`POST /v1/user-required-actions/{id}/resolve`

woocommerce	`DELETE /v1/woocommerce/disconnect`
	`PATCH /v1/woocommerce/settings`
	`POST /v1/woocommerce/orders/bulk-push`
	`POST /v1/woocommerce/orders/{id}/push`

7. Method

- **Target:** production API https://api.lyfflow.com (nginx in front of FastAPI), with the session of a business-admin account on the 'Good one' business (2 Facebook pages, WooCommerce connected, real customer data).
- **Client:** curl 8.18 on Windows 11, run from a developer machine on an ordinary internet connection (≈219 ms round trip to the API). Every number includes this network path, which is why server time is reported separately.
- **Reused-connection runs:** 20 rounds. Each round requests all 58 endpoints one after another over a single keep-alive connection, which spreads load and network noise evenly across endpoints.
- **Fresh-connection (cold) runs:** 3 rounds, one new TCP+TLS connection per request.
- **Server-time estimate:** time to first byte − time to send the request − one round trip (median TCP handshake = 219 ms). It's an outside estimate; a Server-Timing header would make it exact.
- **Percentiles** use linear interpolation. With 20 samples per endpoint, p99 sits next to the max, so the tables show p90, p95 and max instead.
- **Inputs:** real IDs from this business (first page, newest conversation, order, lead, product, comment). Dashboard ranges match the frontend presets (30-day default, 7-day), and query parameters match src/services/api.js.
- **Not covered:** behavior under concurrent load and other tenants' data sizes. Treat these numbers as single-user latency. A load test (k6/Locust) on staging is the next step for capacity.